

Chapter 9. Adding Peripherals Using SPI and I²C

Thirty spokes meet at a nave;Because of the hole we may use the wheel.Clay is molded into a vessel;Because of the hollow we may use the cup.Walls are built around a hearth;Because of the doors we may use the house.Thus tools come from what exists,But use from what does not.

—Lao Tse, *Tao Te Ching*

In this chapter, we'll look at two simple interfaces used to connect peripheral chips to microcontrollers, within a single embedded system. These interfaces allow you to connect devices such as real-time clocks, non-volatile memories for parameter storage, sensor interfaces, and much more. The interfaces are easy to use and cheap to implement, making them ideal for small embedded applications. Some microcontrollers incorporate both types of interface, whereas others may have only one or the other. Which to use really depends on what your processor has to offer and the requirements of the particular peripheral you're using.

Serial Peripheral Interface

The *Serial Peripheral Interface* (known as *SPI*) was developed by Motorola to provide a low-cost and simple interface between microcontrollers and peripheral chips. (SPI is sometimes also known as a *four-wire interface*.) It can be used to interface to memory (for data storage), analog-digital converters, digital-analog converters, real-time clock calendars, LCD drivers, sensors, audio chips, and even other processors. The range of components that support SPI is large and growing all the time.

Unlike a standard serial port (which is covered in [Chapter 10](#)), SPI is a synchronous protocol in which all transmissions are referenced to a common clock, generated by the master (processor). The receiving peripheral

(slave) uses the clock to synchronize its acquisition of the serial bit stream. Many chips may be connected to the same SPI interface of a master. A master selects a slave to receive by asserting the slave's chip select input. A peripheral that is not selected will not take part in a SPI transfer.

SPI uses four main signals: Master Out Slave In (MOSI), Master In Slave Out (MISO), Serial CLock (SCLK or SCK), and Chip Select (CS) for the peripheral. Some processors have a dedicated chip select for SPI interfacing called Slave Select (SS).

MOSI is generated by the master and is received by the slave. On some chips, **MOSI** is labeled simply as Serial In (SI) or Serial Data In (SDI). **MISO** is produced by the slave, but its generation is controlled by the master. **MISO** is sometimes known as Serial Out (SO) or Serial Data Out (SDO) on some chips. The chip select to the peripheral is normally generated by simply using a spare I/O pin of the master. **Figure 9-1** shows a microprocessor interfaced to a peripheral using SPI.

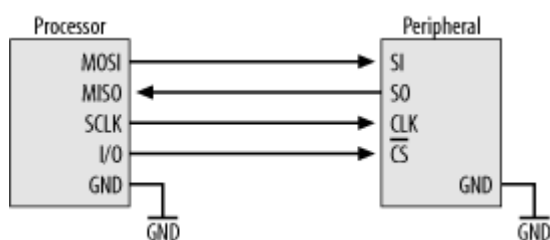


Figure 9-1. Basic SPI interface

Both masters and slaves contain a serial shift register. The master starts a transfer of a byte by writing it to its SPI shift register. As the register transmits the byte to the slave on the **MOSI** signal line, the slave transfers the contents of *its* shift register back to the master on the **MISO** signal line (**Figure 9-2**). In this way, the contents of the two shift registers are exchanged. Both a write and a read operation are performed with the slave simultaneously. SPI can therefore be a very efficient protocol.

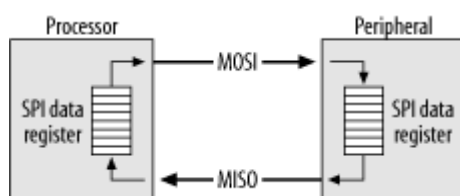


Figure 9-2. SPI transmission

If only a write operation is desired, the master just ignores the byte it receives. Conversely, if the master just wishes to read a byte from the slave, it must transfer a dummy byte in order to initiate a slave transmission.

Some peripherals can handle multiple byte transfers, with a continuous stream of data shifted from the master. Many memory chips with SPI interfaces work this way. With this type of transfer, the chip select for the SPI slave must remain low for the entire duration of the transmission. For example, a memory chip might expect a “write” command to be followed by four address bytes (starting address), then the data bytes to be stored. A single transfer may involve the shifting of a kilobyte or more of information.

Other slaves need only a single byte (for example, a command byte for an analog-digital converter), and some even support being daisy-chained together ([Figure 9-3](#)).

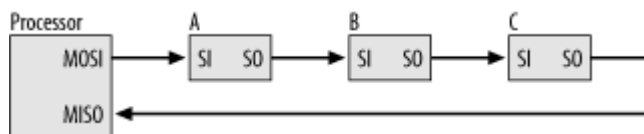


Figure 9-3. Daisy-chaining three SPI devices

In this example, the master processor transmits 3 bytes out of its SPI interface. The first byte is shifted into slave A. As the second byte is transferred to slave A, the first byte is shifted out of slave A and into slave B. Similarly, as the third byte is shifted into slave A, the second byte is shifted into slave B, and the first byte is shifted into slave C. If the master wishes to read a result from slave A, it must again transfer a 3-byte (dummy) sequence. This will move the byte from slave A into slave B, then into slave C, and finally into the master. In the process, the master also receives bytes from slave C and slave B in turn.

Note that daisy-chaining won't necessarily work with all SPI devices, especially ones that require multibyte transfers (such as memory chips). Again, it's a case of checking the slave chips' datasheets carefully to determine what you can and can't do. If the datasheet doesn't explicitly mention daisy-chaining, then it's a fair bet that the device doesn't support it.

SPI has four modes of operation, depending on clock polarity and clock phase. For low clock polarity, the clock (**SCK**) is low when idle and toggles high during a transfer. When configured for high clock polarity, the clock is high when idle and toggles low during a transfer.

The two clock phases are known as *clock phase zero* and *clock phase one*. For clock phase zero, **MOSI** and **MISO** outputs are valid on the rising edge of the clock (**SCK**) if the clock polarity is low ([Figure 9-4](#)). If the clock polarity is high, these outputs are valid on the falling edge of **SCK**, for clock phase zero ([Figure 9-5](#)). The X bit output on **MISO** is an undefined extra bit and is a consequence of the SPI interface. You don't need to worry about it as the SPI interfaces ignore it.

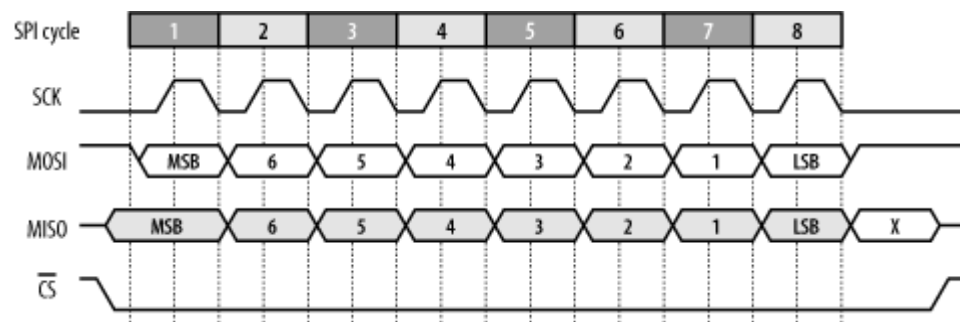


Figure 9-4. SPI timing with clock polarity low and clock phase zero

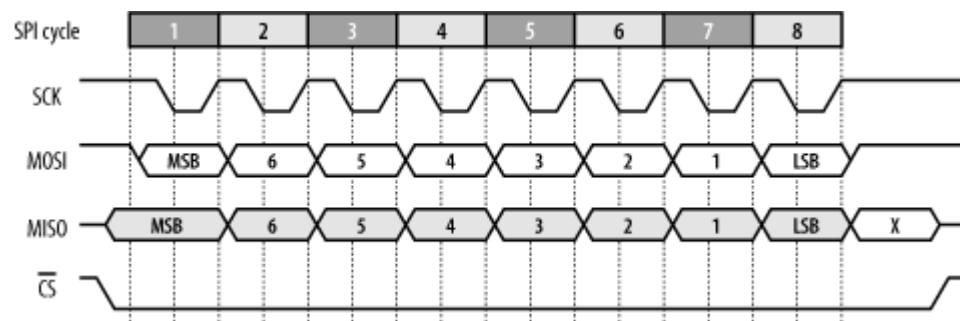


Figure 9-5. SPI timing with clock polarity high and clock phase zero

Conversely, for clock phase one, the opposite is true. **MOSI** and **MISO** are valid on the falling edge of the clock if clock polarity is low ([Figure 9-6](#)). They are valid on the rising edge of the clock if the clock polarity is high ([Figure 9-7](#)).

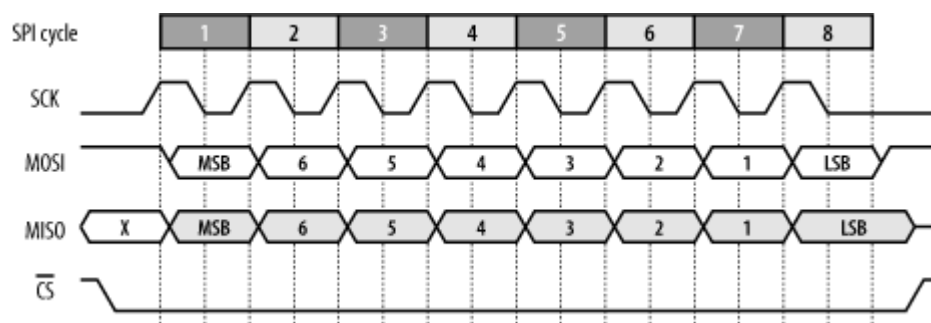


Figure 9-6. SPI timing with clock polarity low and clock phase one

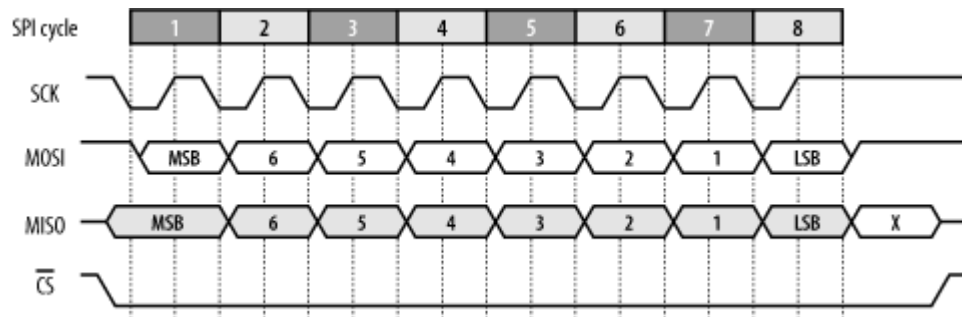


Figure 9-7. SPI timing with clock polarity high and clock phase one

SPI-based Clock/Calendar

There is a wide variety of SPI devices available, and we'll be looking at several in the coming chapters. In the meantime, to see how a SPI interface is used to add a peripheral to a microcontroller, let's look at interfacing a processor to a clock-calendar chip. Such chips contain an oscillator module driven by a crystal, just like a processor. The oscillator module ticks over internal counters that track milliseconds, seconds, minutes, hours, days, months, and years. They are specifically designed to provide accurate timekeeping, and many have additional functions such as an "alarm" (whereby the processor is interrupted at a specific time) and watchdog functions. Some also include voltage monitoring, so that the clock chip may act as a system monitor, alerting the processor should the power supply waver. A number of clock chips are available (and not all are interfaced using SPI). For this example, we will use the Maxim DS1305.

The way in which we interface the clock chip to a processor is virtually identical for all other SPI devices. Some chips with SPI interfaces have special requirements, but most are very simple and straightforward. This

makes SPI a very useful interface that makes increasing system functionality trivial.

The Maxim DS1305 *Real-Time Clock (RTC)* provides timekeeping services and tracks seconds, minutes, hours, day of the month, month, day of the week, and year. It knows which months have 30 days and which have 31. It even automatically adjusts for leap years, up to the year 2100. It can generate two interrupts to the microcontroller for time-of-day alarms. These alarms can be used to trigger a regular system event, such as a backup or user notification.

The DS1305 can run off two separate power sources and supports battery backup of its internal state. The chip can use a power supply in the range 2V to 5.5V, allowing it to be powered from a variety of sources. It also has 96 bytes of static RAM, used for parameter storage. You could use the RAM for holding variables indicating system mode, secure password storage, or even authorization codes for your embedded software, just as desktop software does.

The RAM, like the timekeeping function, is battery backed, and so its contents will be retained for the life of the battery. This can be up to 10 years, depending on the battery chosen. Thus, the contents of the internal parameter RAM will probably last for the expected operational life span of an embedded system.

TIP

If you are producing commercial embedded systems and have problems with late-paying customers, you can use this RAM to hold a license number. When you ship the system, you design it to work for perhaps 45 days before shutting down. When your customer pays her bill, and you supply her with the right magic number, the system comes back to life again. The system stores the license number in the RAM of the RTC and from then on works as normal.

The DS1305 is versatile in the way it can be powered. It has three power-supply inputs (**VCC1**, **VCC2**, and **VBAT**) from which it can choose to draw power. **VCC1** is the primary supply input and is connected directly to the system power supply. When the computer is up and running, the DS1305 draws its current from this source. **VCC2** is the secondary power source, and this can be a rechargeable battery. **VBAT** is the third power source and is for nonrechargeable batteries.

There are three, and only three, possible configurations for powering the DS1305, and it is important for correct operation that the power inputs are appropriately driven. **Figure 9-8** shows the DS1305 powered by a primary DC supply connected to **VCC1** and a secondary, nonrechargeable battery connected to **VBAT**. (To keep the diagram simple, only the power pins are shown. We'll look at the data interface in a moment.) For this configuration, **VCC2** is unused and must be connected to **GND**. When **VCC1** falls below a given threshold voltage (the primary power source has failed), the internal memory and registers of the DS1305 become write protected to prevent their being corrupted by a failing microprocessor.

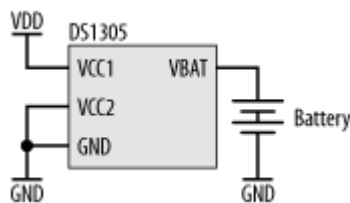


Figure 9-8. Using the DS1305 with a nonrechargeable battery

If the secondary power source is a rechargeable battery, then the DS1305 may be wired as shown in **Figure 9-9**. When using a rechargeable battery on **VCC2**, **VBAT** must be connected to **GND**. When the device is used in this mode, there is no automatic write protection for the DS1305 if **VCC1** fails.

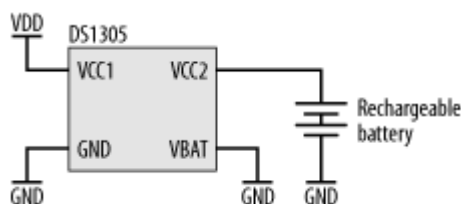


Figure 9-9. Using the DS1305 with a rechargeable battery

Finally, the DS1305 may be used with only a battery as its primary power source and no backup power supply. This is shown in [Figure 9-10](#). For this configuration, both **VCC1** and **VBAT** are connected to ground, while the battery is connected to **VCC2**.

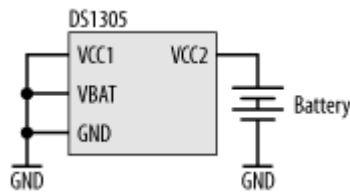


Figure 9-10. Using the DS1305 with a battery as its only power source

Using the DS1305 is very simple. The schematic showing a DS1305 interfaced to a microcontroller is shown in [Figure 9-11](#).

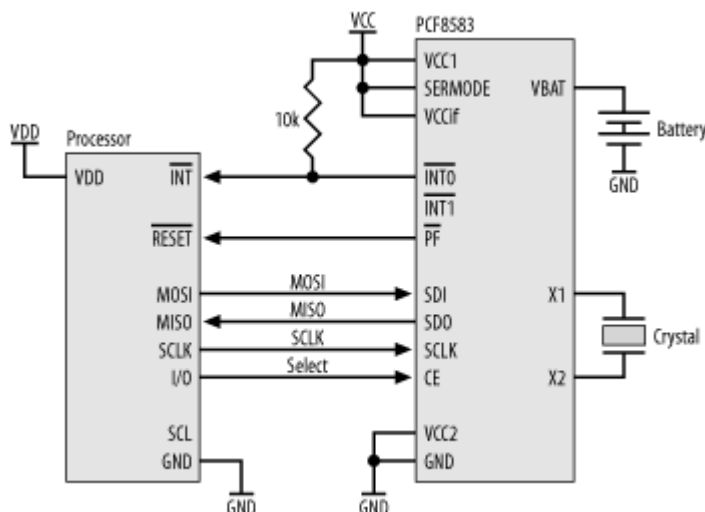


Figure 9-11. A DS1305 RTC interfaced to a microcontroller

The serial interface of the DS1305 can operate as either a SPI port or a three-wire^[13] port. The input **SERMODE** (SERial MODE) selects which serial mode to use. Connecting **SERMODE** to the power supply selects SPI operation. Connecting **SERMODE** to **GND** selects three-wire operation. (For three-wire operation, **SDO** and **SDI** are tied together.) The connection to a microcontroller's SPI port is straightforward, with **MOSI**, **MISO**, **SCLK**, and a chip select, as we've seen previously. There is one important difference, though, for the DS1305. It has a high-active **CE** (Chip Enable), rather than the more common low-active chip selects of other SPI devices. Therefore, the processor's I/O line driving **CE** must be low when the device is not selected and high when the device is selected.

The DS1305 has a special Power Fail (**PF**) output that is asserted low when the primary power source, **VCC1**, falls below the secondary power source (**VCC2** or **VBAT**). This can be used to alert the processor of the power fail (by using it as an interrupt) or to stop the processor (by connecting it to the processor's **RESET**). This is used to prevent a failing processor from corrupting devices as the power dies. If you don't require a power-fail notification, **PF** may be left unconnected.

The input **VCCif** (**VCC** for the interface logic) selects the output voltage levels of **SDO** and **PF**. Since the DS1305 can be used in both 5V and 3.3V systems, this input allows the output levels of these pins to be set to the appropriate high voltage. **VCCif** is just connected to the system's power supply. Thus, for a 5V system, **VCCif** is 5V, and the outputs of the DS1305 are also 5V. Similarly, for a 3.3V system, **VCCif** is 3.3V and so are the outputs.

The DS1305 has two interrupt outputs, **INT0** and **INT1**. These may be used to interrupt the processor when a DS1305 alarm function triggers. As the interrupt outputs are open-drain, each requires a 10k resistor to pull it high when it is inactive. If one or both of the interrupts are not required, just leave them unconnected. Only **INT0** is used in our example, and so **INT1** is safely ignored.

Finally, the DS1305 has two crystal inputs, **X1** and **X2**. A 32.768kHz watch crystal is connected across these pins, providing the timing source for the internal clock.

So that is the DS1305, a versatile little chip that can provide timekeeping for your embedded system. It's easy to use, and the programming information for it is contained in the device's datasheet.

SPI-based Digital Potentiometer

Let's look at another simple SPI example. This time, we will interface a digital potentiometer to a microprocessor. Before getting into the details, let's take a look at what one is and why you'd use it. We'll get back into SPI in just a moment.

A potentiometer (also known simply as a pot , trimmer , or trim pot) is just a variable resistor. The symbol for a pot is shown in [Figure 9-12](#). Pots are normally mechanical components and are manually adjusted. Your stereo probably uses pots for its volume, bass, and treble controls. The brightness and contrast knobs for monitors and LCDs are also potentiometers.

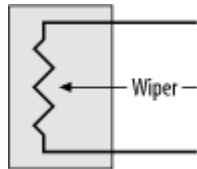


Figure 9-12. A potentiometer

A standard potentiometer consists of two terminals (the upper and lower pins in the diagram) that connect to either end of a resistor. A third terminal, known as the wiper , moves up or down the resistor, effectively tapping into the voltage present at a given point. Move the wiper one way, and the amount of resistance the wiper sees is increased. Move it the other way, and the resistance decreases. Mechanical pots come in a variety of resistance ranges, and their accuracy is not particularly good. They may be used to provide an adjustable voltage output ([Figure 9-13](#)) or simply to vary the resistance used in an analog circuit.

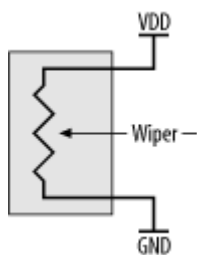


Figure 9-13. Using a potentiometer to provide a variable voltage between VDD and ground

As a simple example, you could use a pot to vary the intensity of a LED, as shown in [Figure 9-14](#). Here, the fixed resistance between the LED's anode and the pot's wiper is 300Ω . By adjusting the wiper, we add to this resistance, thus decreasing the current flow through the LED and reducing its brightness.

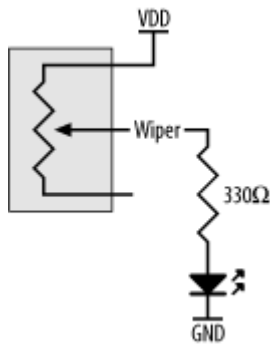


Figure 9-14. Using a potentiometer to vary the intensity of a LED

Note how one terminal of the pot is unconnected. This is fine, since in this case we are not using the pot to provide an intermediate voltage between two values. Rather, we are simply using the pot as a variable resistor, increasing the impedance between the wiper and **VDD**.

Now, a standard pot is manually adjusted. It either will have a knob attached (as in a volume control or brightness adjustment) or will have a small notch for screwdriver adjustment. Wouldn't it be great if your microprocessor could adjust the pots in your analog circuits, under software control? That way, your application software could adjust the brightness of the display or change the volume of the sound system. Well, by using a digital potentiometer, you can do just that. Televisions, computer monitors, and stereos with internal embedded controllers use digital pots to adjust settings such as volume. When you hit a volume button on a remote control, the TV or stereo adjusts the settings of digital pots, which are part of the amplifiers driving the speakers.

Figure 9-15 shows an Analog Devices AD5203 digital potentiometer with a SPI interface. This chip has four potentiometers, all of which may be adjusted under software control. Each pot has a possible 64 positions, and versions of the chip are available with either 10kΩ or 100kΩ impedances. For higher resolution, the pin-compatible AD8403 has a possible 256 settings, also configurable through a SPI interface.

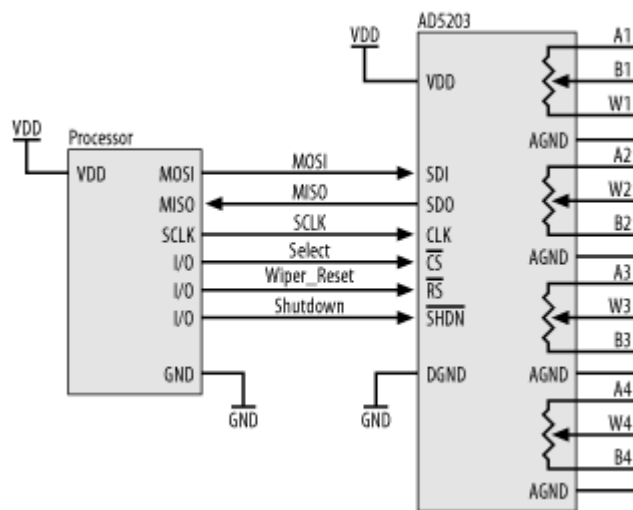


Figure 9-15. Interfacing a digital potentiometer to a processor using SPI

The AD5203 has a Serial Data Input (**SDI**), which is connected to the processor's **MOSI** output. Similarly, the device's Serial Data Output (**SDO**) is connected to **MISO**. The AD5203's clock input (**CLK**) is positive-edge-triggered midway through each SPI cycle, which means that any processor communicating with it must use high clock polarity and clock phase one on **SCLK**. The Chip Select (**CS**) of the AD5203 may be driven by a processor digital I/O line. The AD5203 has two other inputs, Shutdown (**SHDN**) and Reset (**RS**). **SHDN** places the device in low-power mode, and **RS** resets the potentiometer wipers to their midpoint position. Both of these inputs may also be driven by a processor I/O line, or if their functionality is not needed, they may be simply tied high using 10k Ω pull-up resistors.

The potentiometers within the AD5203 are used as any other pots would be. The **A** and **B** terminals connect to either end of the internal resistors, and the position of the wiper (**W**) is adjusted under software control.

The AD5203 has several ground connections. **DGND** is the digital ground for the SPI interface and control logic of the chip. **AGNDs** are the analog grounds of the internal potentiometers and should all be connected to **DGND** at a single point.

The datasheet for the AD5203 provides the control codes needed to configure the chip, and its use is simple and straightforward.

Adding Nonvolatile Data Memory with SPI

The internal memory of microcontrollers is very small, and their data storage capabilities are severely limited. We're now going to look at how you can increase the storage capacity of your embedded system by adding an ATMEL AT45DB161 2M serial DataFlash using SPI. These chips are commonly used in low-cost digital cameras and answering machines. You could also use this flash chip as a virtual disk drive in your embedded system.

Most other flash chips have a bus interface, but the AT45DB161 has a serial interface, making it well-suited for use with small microcontrollers. The AT45DB161 is a 2M chip, but you can get similar ATMEL chips in capacities ranging from 512K to 8M. They all use the same physical interface, so the same design works for all. (Note, however, that their pinouts and physical packages vary, so one chip will not mount onto a circuit board designed for another.)

The chip consists of an array of flash memory, organized as individual pages of 528 bytes each, and two RAM buffers, also 528 bytes each ([Figure 9-16](#)). To write data into the main flash array, the processor must first write data into one of the buffers and then issue a command to write that buffer into the array. A processor can read the contents of either of the buffers, transfer a flash page to the buffers, or read from the flash array directly. The operation of the buffers is independent, and one buffer may be accessed by the processor (via SPI) while the contents of the other buffer are being written into the flash array.

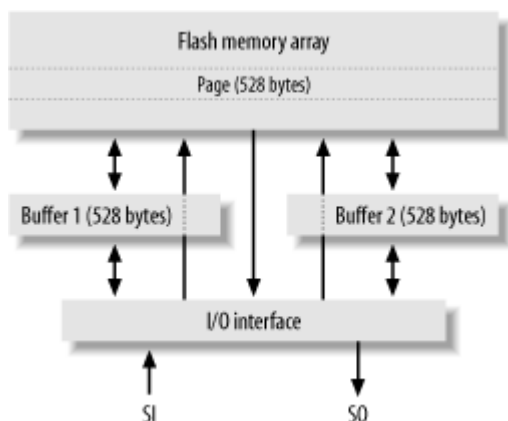


Figure 9-16. AT45DB161 internal architecture

The flash supports numerous commands for writing to and reading from the buffers, writing the buffers to the main array and transferring an array page back to a buffer. The ATMEL datasheet has full details of the software protocols and command set.

There are a few things to note about the internal architecture and the flash array. The first is that one 528-byte page of the flash array is not contiguous with the next. In other words, if you are using a pointer in your software to track the current location in the memory, you can't just increment it from the end of one page and expect it to be pointing to the next. Every 528 bytes (and it's a strange number), you have to leap forward to the next page. Think of it as pages of 528 bytes with big gaps in between.

The second catch with this memory is that it only has a lifetime of 1000 write cycles per page. Most flash technologies (and there are several different types) support 100,000 write cycles or better, and you can normally exceed this limit and the device will keep working reliably for you. This isn't the case with the AT45DB161. Once the 1000-write limit is exceeded, memory locations start failing on you. The chip will read existing data back correctly, but new pages will not write successfully. Depending on the application, this limit may not be a problem. My company uses this particular chip in our long-term dataloggers. These machines are deployed for year-long deployments, collecting (and compressing) data and storing it away in the flash chip. The logger gradually builds a page image in one of the buffers before storing it to the array in a single write. Since, during a deployment, a page will be written only once (and then the logger will move on to the next page), the 1000-write limitation isn't a problem. It would take a thousand deployments before the chip would fail. However, if you are using the chip for variable storage and are modifying the flash pages on a byte-by-byte basis, you're in trouble. Individually changing 528 bytes within a page counts as 528 writes. So do that twice to a page, and suddenly you're over the limit. Therefore, this flash is well-suited to some applications and not others.

The basic design for using an AT45DB161 is shown in [Figure 9-17](#).

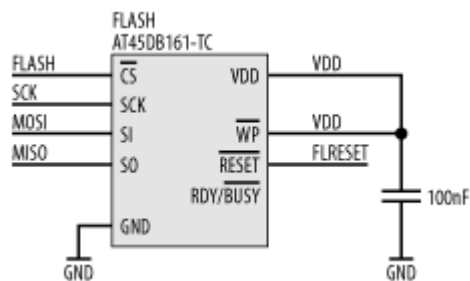


Figure 9-17. 2M serial DataFlash

On the left of the chip are the SPI interface connections, **MOSI**, **MISO**, and **SCK**, and a chip select (**FLASH**). The chip will support SPI transfers at up to 20MHz, so the SPI interface can be run very fast indeed. On the right of the chip is the power supply, **VDD**, which is decoupled to ground using a 100nF capacitor. The AT45DB161 requires a power supply in the range of 2.5V to 3.6V. However, its logic inputs are 5V tolerant, meaning that this chip can be used in systems with mixed power supplies. In other words, while this chip requires a 3V power supply, it can be directly interfaced to a processor with a 5V supply (and 5V logic levels). The AT45DB161 has a write-protect pin (**WP**), which, when driven low, prevents the contents of the flash being modified. If you don't require write protection, simply tie this input high, as shown in the schematic. The flash also has a **RESET** input so that the chip can be manually reset under software control. The flash incorporates a built-in power-on reset that will put the device into a known state; therefore, a "manual" reset at power-up should be unnecessary. However, I've found that the internal power-on reset generator is somewhat finicky and doesn't always kick in as it should. Under such circumstances, the flash fails to enter a known state and is unusable in the system. Therefore, I have found it good practice to give the processor control of the flash's reset. As part of the processor initialization routines executed in its reset firmware, I get the processor to reset the flash, nudging it into reality. It's a simple thing, but makes all the difference for a reliable system. Pin 1 is a status output (**RDY/ BUSY**) indicating whether the device is ready or if it is still completing an internal operation.

The connections for interfacing this memory chip to an ATMEL 90S4434 AVR processor are shown in [Figure 9-18](#). The AVR portion of the schematic is no different from the examples we have seen previously. That's the nice thing about simple interfaces such as SPI. They form little subsystem modules that "bolt together" like building blocks. Start with

the basic core design, and just add peripherals as you need them. The schematic also shows decoupling capacitors for the power supplies, the crystal oscillator for the processor, and a pull-up resistor for **RESET**. Pin 41 (**PB1**) is used as a “manual” (processor-controlled) reset input to the flash.

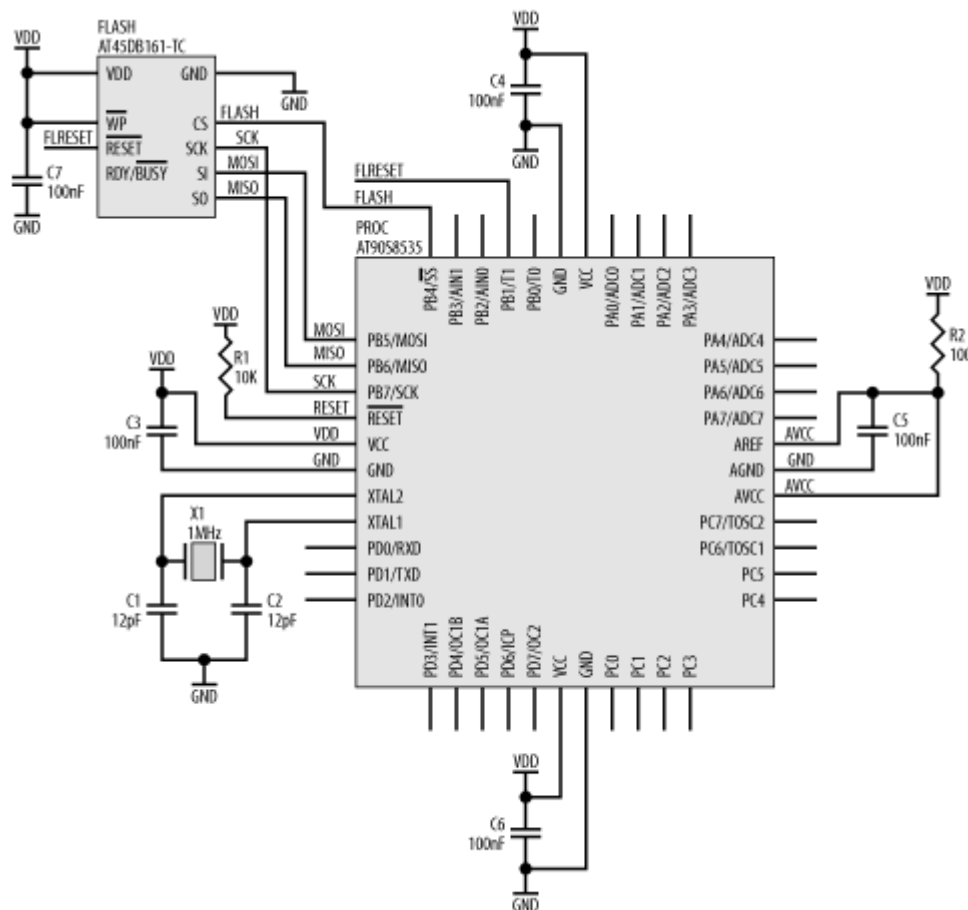


Figure 9-18. A 2M DataFlash interfaced to an AT90S4434

Adding a Parameter Memory Using SPI

We saw in the previous section how to add a large-capacity serial flash for data storage. Using nonvolatile memory to hold system parameters can be valuable as a way of preserving important key variables during periods of no power. But the AT45DB161 DataFlash is just not the device for that task. It is better suited to data recording, and its large capacity is overkill for parameter storage. So, now we’re going to look at how you can use SPI to add a small parameter memory (in the form of an EEPROM) to your embedded system. The EEPROM I’ve chosen is the ATMEL AT25640. This device will hold data for at least 100 years without power and will endure more than one million write cycles (significantly more than an AT45DB161!). In that way, your software can happily alter

parameter variables without fear of limiting the life span of the chip. The AT25640 has only 8K of memory, which might not sound like much. But don't forget, that's 8192 char variables, which is more than enough storage space for most parameters. If 8K is too much, there are also versions of the chip with 1K (AT25080), 2K (AT25160), and 4K (AT25320) bytes of memory.

The architecture and use of the AT25640 is much simpler than that of the AT45DB161. Full details of the required software protocol are in the ATMEL datasheet for this chip.

The schematic for an AT25640 circuit is shown in [Figure 9-19](#).

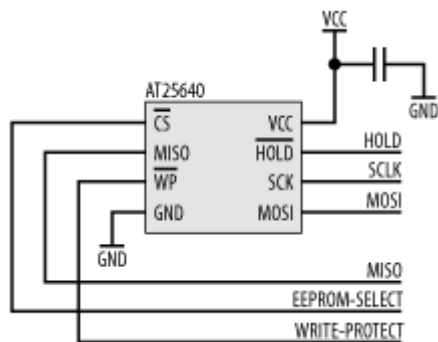


Figure 9-19. Using an AT25640 EEPROM

The interface is standard SPI, and the chip also has a write-protect input and a hold input. Asserting hold allows the processor to temporarily stall a serial transfer (while it performs other tasks) without terminating the access to the AT25640. And, as you might expect, write-protect, when asserted, turns the chip into a read-only device. These control inputs may be driven by programmable I/O lines of the processor. The only other requirement is power (which is decoupled to ground using a 100nF capacitor) and ground. The chip is available in two types. One will operate from a supply voltage of between 2.7V and 5.5V, while the other needs a supply voltage of between 1.8V and 3.6V.

[13] Developed by National Semiconductor, three-wire, also known as MicroWire, is very similar to SPI and is found in some microcontrollers and DSP processors. Unlike SPI, which has separate data lines for reading and writing, three-wire uses a common bidirectional data line.

Inter Integrated Circuit

The I^2C (Inter Integrated Circuit) bus is a very cheap, yet effective, network used to interconnect peripheral devices within small-scale embedded systems. It is sometimes also known as IIC and has been in existence for more than 20 years. It is the equivalent of SPI, but its operation is somewhat different.

I^2C uses two wires to connect multiple devices in a multidrop bus. The bus is bidirectional, low-speed, and synchronous to a common clock. Devices may be attached or detached from the I^2C bus without affecting other devices. Several manufacturers, such as Microchip, Philips, Intel, and others produce small microcontrollers with I^2C built in. The data rate of I^2C is somewhat slower than SPI, at 100kbps in standard mode and 400kbps in fast mode.

The two wires used to interconnect with I^2C are **SDA** (serial data) and **SCL** (serial clock). Both lines are open-drain.^[14] They are connected to a positive supply via a pull-up resistor and therefore remain high when not in use. A device using the I^2C bus to communicate drives the lines low or leaves them pulled high as appropriate. Each device connected to the I^2C bus has a unique address and can operate as a transmitter (a bus master), a receiver (a bus slave), or both (**Figure 9-20**). I^2C is a multimaster bus, meaning that more than one device may assume the role of bus master.

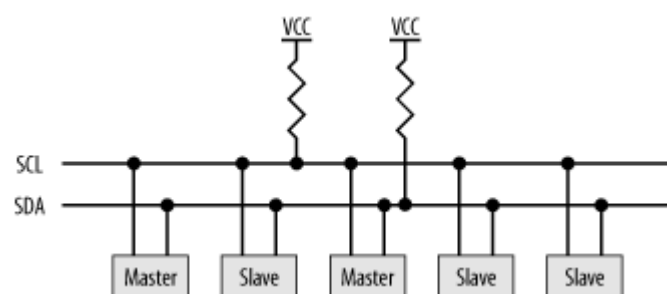


Figure 9-20. I^2C block diagram

Both **SDA** and **SCL** are bidirectional. Unlike SPI, which has separate data lines for each direction of communication, I^2C shares the same signal line

for master transmission and slave response. Also unlike SPI, I²C does not have several modes of operation. The timing relationship between the clock, **SCL**, and the data line, **SDA**, is simple and straightforward. When idle, both **SDA** and **SCL** are high. An I²C transaction begins with **SDA** going low, followed by **SCL** (**Figure 9-21**). This indicates to all receivers on the bus that a packet transmission is commencing. While **SCL** is low, **SDA** transitions (high or low) for the first valid data bit. This is known as a **START condition**.

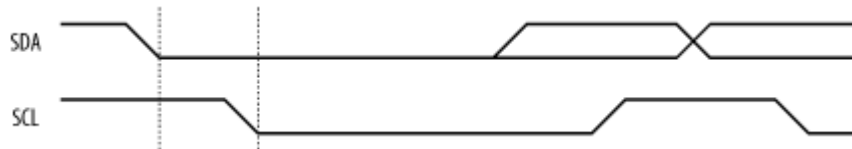


Figure 9-21. Start of packet

For each bit that is transmitted, the bit must become valid on **SDA** while **SCL** is low. The bit is sampled on the rising edge of **SCL** and must remain valid until **SCL** goes low once more. Then **SDA** transitions to the next bit, before **SCL** goes high once more (**Figure 9-22**).

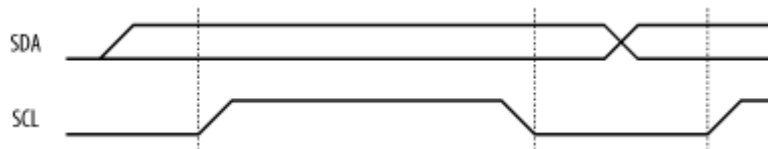


Figure 9-22. Timing relationship between SDA and SCL

Finally, the transaction completes by **SCL** returning high (inactive) followed by **SDA** (**Figure 9-23**). This is known as a **STOP condition**.

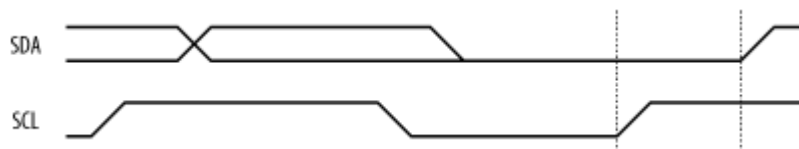


Figure 9-23. End of packet

Any number of bytes may be transmitted in an I²C packet. As with SPI, the most significant bit of the packet is transmitted first. If the receiver is unable to accept any more bytes, it can abort the transmission by holding **SCL** low. This forces the transmitter to wait until **SCL** is released again.

Each byte transmitted must be acknowledged by the receiver. Upon the transmission of the eighth data bit, the master releases the data line **SDA**. The master then generates an additional clock pulse on **SCL**. This triggers the receiver to acknowledge the byte by pulling **SDA** low (**Figure 9-24**). If the receiver fails to pull **SDA** low, the master aborts the transfer and takes appropriate error-handling measures.



Figure 9-24. I2C packet with receiver acknowledge

Now, I²C is a multimaster bus. So, more than one master may attempt to start transmission at the same time. Since the bus's default state is high, a master transmitting a 0 bit will pull **SDA** low but will leave the bus in its default state if the bit is to be a 1. Thus, if two masters begin simultaneous transmission, a master leaving the bus in its default state for a 1 bit, but detecting the bus pulled low by another master (for a 0 bit), will register an error condition and abort the transmission.

Now, SPI used a separate chip select to enable a receiving slave. Each SPI slave has a separate chip select, generated by the master. I2C does not have such a selection mechanism. Instead, each device on the bus has a unique address, and the packet transmission begins with address bits, followed by the data. An address byte consists of 7 address bits, followed by a direction bit. If the direction bit is a 0, the transmission is a write cycle and the selected slave will accept the data as input. If the direction bit is a 1, then the request is for the slave to transfer data back to the master. An example packet, transferring 1 byte of data, is shown in **Figure 9-25**.

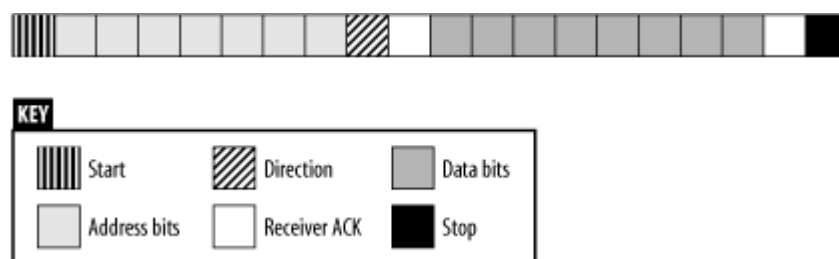


Figure 9-25. An I²C packet

A special address, known as the `general call address`, broadcasts to all I²C devices. This address is `%0000000` with a direction bit 0. The general call is the mechanism by which the master determines what slaves are available, and there are several types of general call. The second byte of a general call indicates the purpose of the general call to the slaves. Upon receiving the second byte, individual slaves will determine whether the command is applicable to them, and if so they will acknowledge. If the command is not applicable to a given slave, then the slave simply ignores the general call and does not acknowledge. If the second byte is `0x06` (`%00000110`), then this indicates that appropriate slaves should reset and respond with their addresses. If the second byte is `0x04` (`%00000100`), slaves respond with their addresses but do not reset. Any other second byte of a general call, where the least-significant bit is a 0, should be ignored.

If the least-significant bit of the second byte is a 1, then the general call is by a master device identifying itself to other masters in the system by transmitting its own address. The other bits of the second byte contain the master's address.

Another special address byte is known as the `START byte`. This byte is `%00000001` (`0x01`). It is used to indicate to other masters that a long data transfer is beginning. This is particularly important for masters that do not have dedicated I²C hardware and must monitor the bus by software polling. When a master detects a `START` byte generated by another master, it can reduce its polling rate, allowing it more time for other software tasks.

I²C also supports an extended 10-bit addressing mode, allowing up to 1024 peripherals. Devices that use 7-bit addressing may be mixed with 10-bit addressing devices in a single system. In 10-bit addressing, 2 bytes are used to hold the address. If the (first) address byte begins with `%11110XX`, then a 10-bit address is being generated. The 2 least-significant bits of the first byte, combined with the 8 bits of the second byte, form the 10-bit address ([Figure 9-26](#)); 7-bit devices will ignore the transaction.

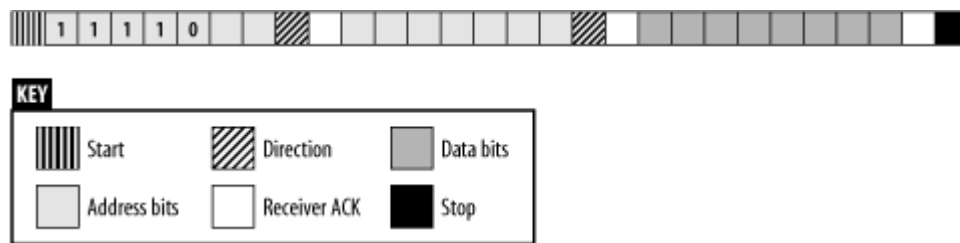


Figure 9-26. An I2C packet with 10-bit addressing

[14] An open-drain or open-collector pin has output drivers that can only pull the signal line to ground. They cannot drive it high. This has the advantage that more than one device connected to a signal line may pull it low. If this were not the case, one device attempting to pull the line low while another tried to pull it high would result in a short circuit, with disastrous results. Interrupt lines are typically open-collector. All open-collector signals need a pull-up resistor and are low active. The idle state (when no device is asserting) is to be pulled high by the resistor.

Adding a Real-Time Clock with I²C

We saw in the previous section how to interface a Real-Time Clock (RTC) to a microprocessor using a SPI interface. Now let's look at how we'd do the same using the I²C interface. For this example, we'll use the tiny Philips PCF8583. It also has 240 bytes of RAM, which, like the DS1305's, may be used for parameter storage. Unlike the DS1305, the PCF8583 does not have an integrated battery-backup system. So, you would need to provide an external battery-backup circuit. Many other I²C RTCs are available, and some do incorporate battery-fail protection. I've chosen to look at this one because it makes for a very simple example of an I²C interface.

The PCF8583 has two pins (**OSCI** and **OSCO**) for connecting a 32.768kHz watch crystal. This crystal pulses an internal circuit that performs the timekeeping functions. The address pin, **A0**, determines the address of the device on the I²C bus. Most I²C chips provide several address pins, allowing a range of possible addresses to be wired. The PCF8583 has only one, to reduce the pin count of the chip. Six of its address bits are hardwired internally. Only the least significant, **A0**, is available to the system designer. The address configuration of the PCF8583 is shown in [Figure 9-27](#). Note how the transfer direction (read or write) is incorporated into the address field.

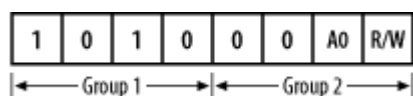


Figure 9-27. PCF8583 addresses

Connecting **A0** directly to ground sets that address bit to 0 and therefore maps the PCF8583 to I²C address 0x50. Alternatively, if **A0** is tied to **VDD**, then the address of the device is 0x51.

The schematic for interfacing the PCF8583 to a microcontroller is shown in [Figure 9-28](#).

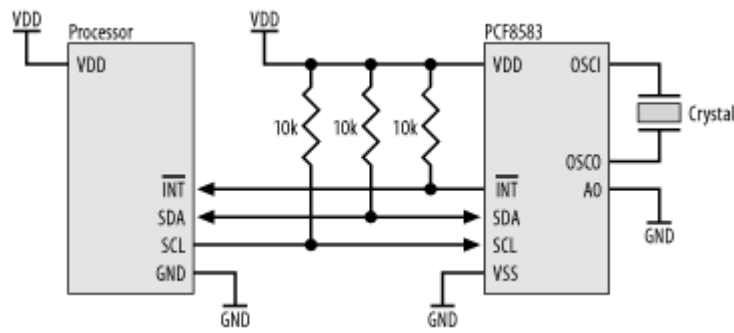


Figure 9-28. Interfacing a PCF8583 to a microcontroller

Both **SDA** and **SCL** require pull-up resistors to **VDD**. The PCF8583 also has an internal alarm function and asserts an output (**INT**) for interrupting the processor. Since this output is open-drain, a pull-up resistor is also required.



Adding a Small Display with I²C

You can use I²C to add simple LCDs (and other equivalent display technologies) to your embedded computer. These LCDs are usually just a few lines of text high but are useful for simple message display functions. Matrix Orbital (<http://www.matrix-orbital.com>) produces a number of display modules that are easy to interface, such as the VFD2041. This display module is 80 characters wide by four lines deep. The interface circuit is shown in **Figure 9-29**, and as you can see, there's almost nothing to it. The types of LCDs found in laptops are considerably more complicated, and interfacing them to small processors is just not an option. But for simple message displays (such as on the front panel of an appliance), a circuit like this is ideal.

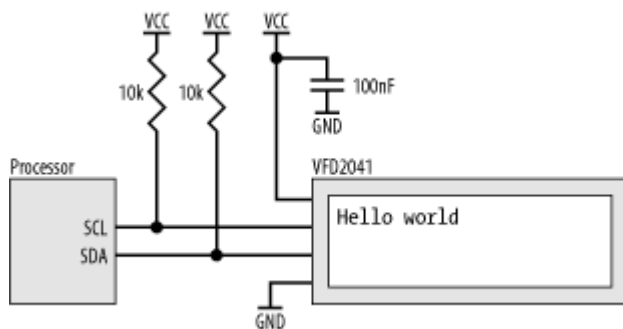


Figure 9-29. Interfacing a VFD2041 display using I²C

Many Matrix Orbital displays also come with RS-232C interfaces, so if your embedded processor doesn't support I²C, it's still easy to add a small display.

In the next chapter, we'll see how to add a serial port to an embedded computer and connect it to both external host computers and peripherals. We'll also look at several forms of serial interface, including RS-232C, IrDA and USB.

